

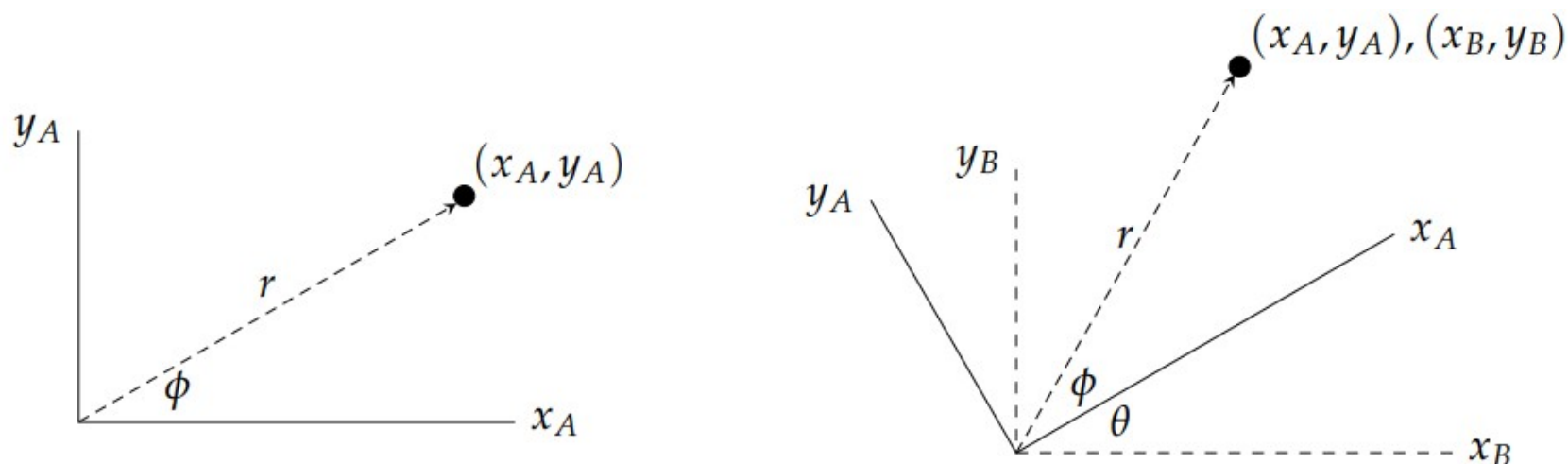
座標轉換，矩陣運算，感測器訊號融合

第三週

Oct/16/2026

中原大學電機系

平面座標轉換及矩陣運算



$$x_A = r \cos \phi$$

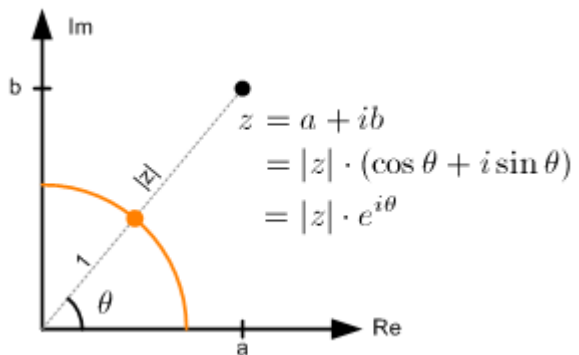
$$y_A = r \sin \phi,$$

$$\begin{bmatrix} x_B \\ y_B \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_A \\ y_A \end{bmatrix}$$

$$\begin{aligned} x' &= r \cos(\phi + \theta) \\ &= r(\cos \phi \cos \theta - \sin \phi \sin \theta) \\ &= (r \cos \phi) \cos \theta - (r \sin \phi) \sin \theta \\ &= x \cos \theta - y \sin \theta, \end{aligned}$$

$$\begin{aligned} y' &= r \sin(\phi + \theta) \\ &= r(\sin \phi \cos \theta + \cos \phi \sin \theta) \\ &= (r \sin \phi) \cos \theta + (r \cos \phi) \sin \theta \\ &= y \cos \theta + x \sin \theta \\ &= x \sin \theta + y \cos \theta. \end{aligned}$$

平面座標轉換及複數表示



$$\begin{aligned} z &= a + ib \\ &= |z| \cdot (\cos \theta + i \sin \theta) \\ &= |z| \cdot e^{i\theta} \end{aligned}$$

$$i = \sqrt{-1}$$

$$i \bullet i = -1$$

$$e^{ix} = \cos x + i \sin x$$

$$z' = |z'| e^{i(\theta+\phi)}$$

$$= |z| e^{i(\theta+\phi)} \quad (\because |z'| = |z|)$$

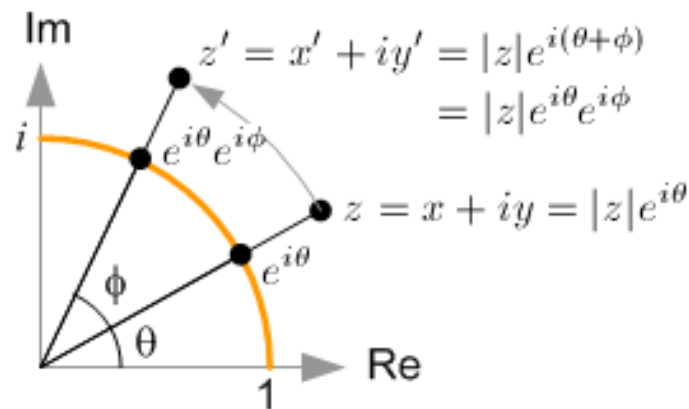
$$= |z| e^{i\theta} e^{i\phi}$$

$$= (x + iy) e^{i\phi} \quad (\because z = |z| e^{i\theta} = x + iy)$$

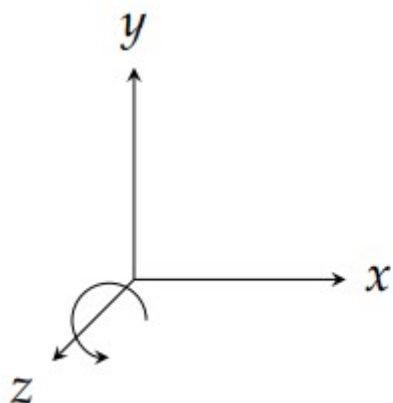
$$= (x + iy)(\cos \phi + i \sin \phi)$$

$$= \cos \phi \cdot x - \sin \phi \cdot y + i(\sin \phi \cdot x + \cos \phi \cdot y)$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

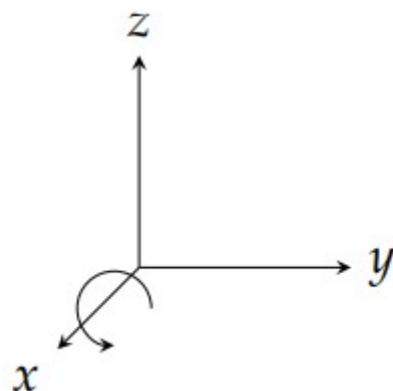


空間座標轉換及矩陣運算



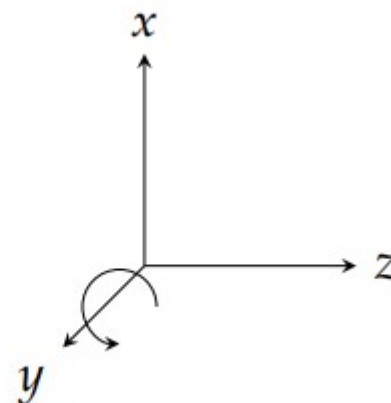
rotate from x
to y around z

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



rotate from y
to z around x

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$



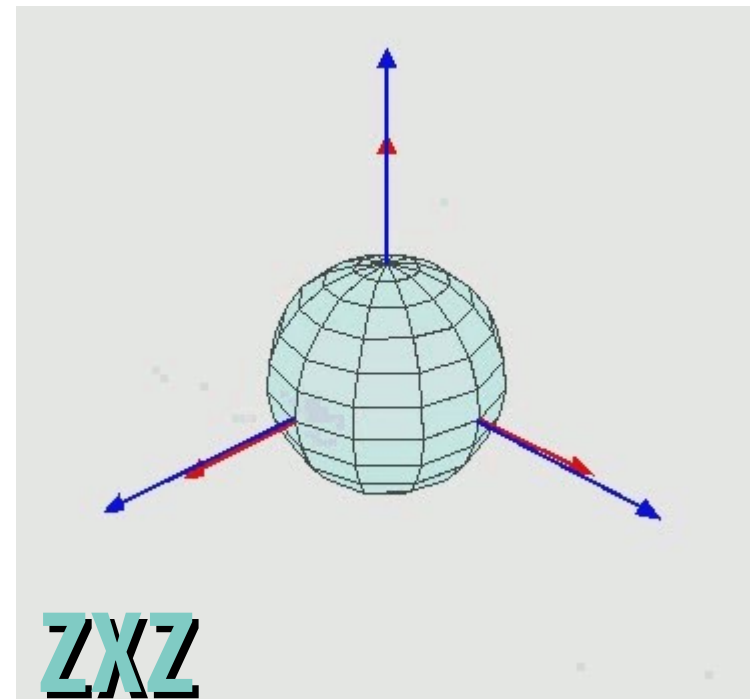
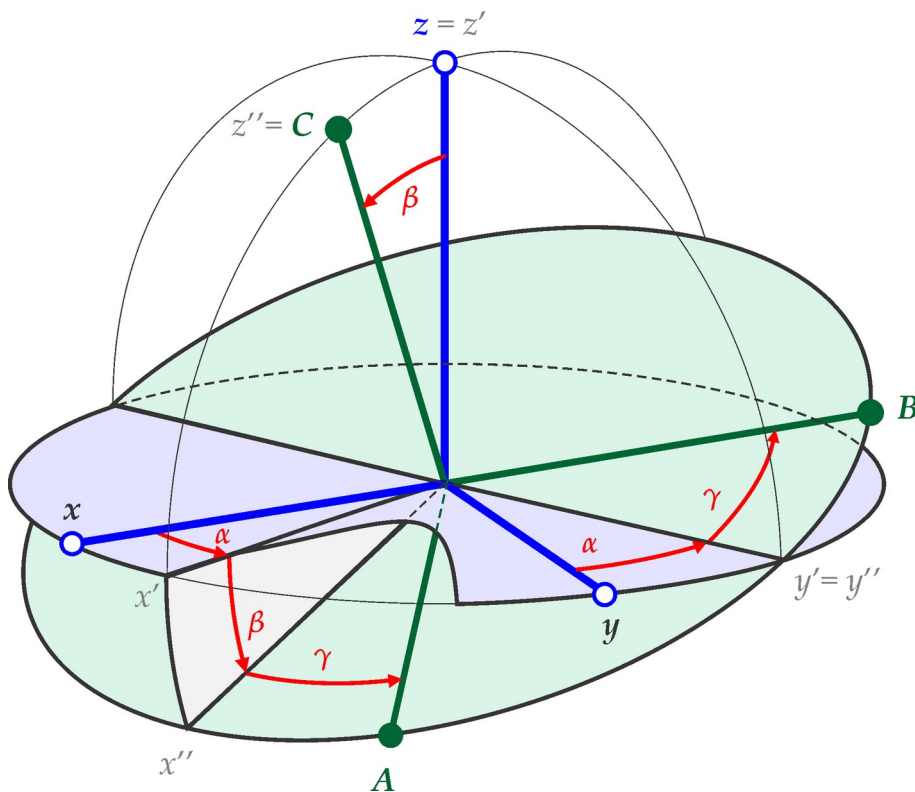
rotate from z
to x around y

$$\begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

矩陣運算和順序有關

有 12 種可能的 Euler 轉動順序，來實現座標轉換

xyx xyz xzx xzy yxy yxz yzx yzy zxy **zxz** zyxx zyz



<https://blender.stackexchange.com/questions/220691/rotate-with-the-z-x-z-euler-sequence-with-animation-nodes>

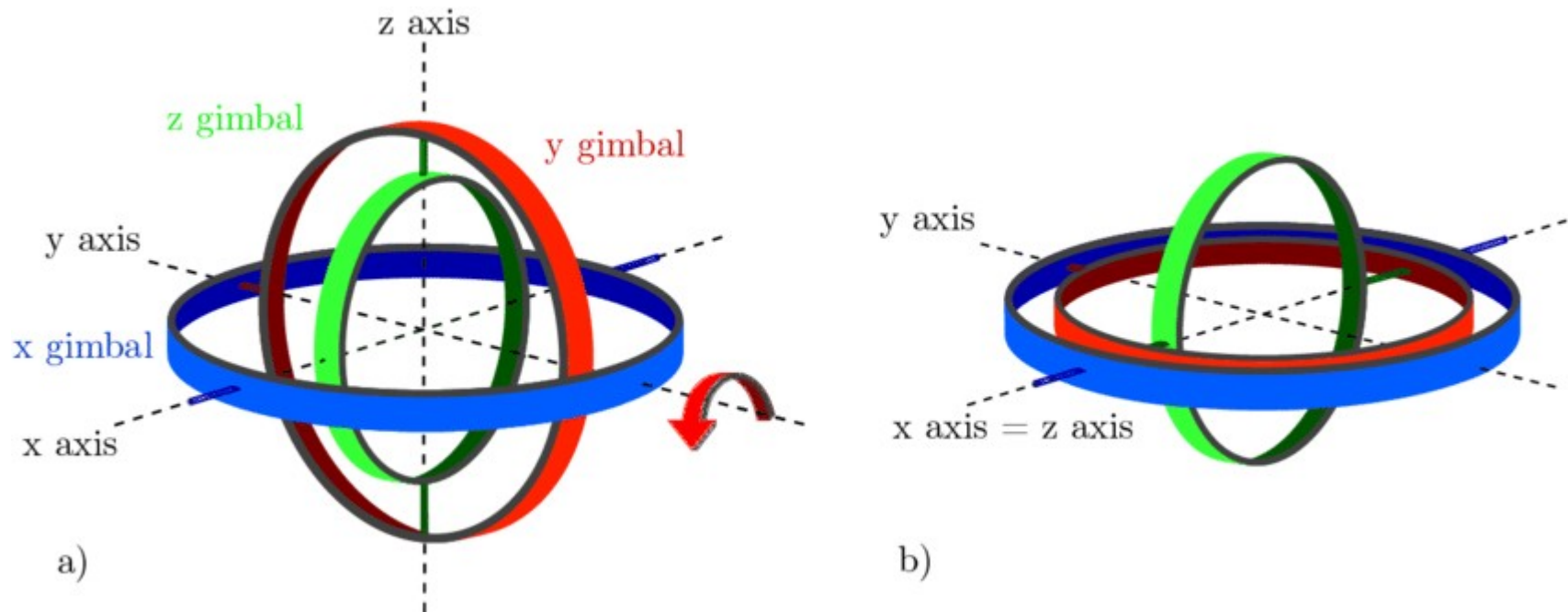
ZYX 旋轉矩陣

$$R_{y(\theta)} R_{x(\phi)} = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \sin \phi & \sin \theta \cos \phi \\ 0 & \cos \phi & -\sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix}$$

$$R_{z(\psi)y(\theta)x(\phi)} = R_{z(\psi)} (R_{y(\theta)} R_{x(\phi)}) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta \sin \phi & \sin \theta \cos \phi \\ 0 & \cos \phi & -\sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix}$$

$$R_{z(\psi)y(\theta)x(\phi)} = R_{z(\psi)} (R_{y(\theta)} R_{x(\phi)}) = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix}$$

Euler 旋轉的困境 - Gimbal Lock



Gimbal Lock 的原理：藍色框 x 軸，紅色框 y 軸，綠色框 z 軸。在圖 a) 中，所有軸都互相垂直。繞 y 軸 旋轉 90 度後，藍色框和綠色框具有相同的旋轉軸如 b) 所示。這種情況妨礙隨後繞 x 或 z 軸旋轉時，清楚地確定旋轉軸。

Gimbal Lock - 從數學來看

Loss of a degree of freedom with Euler angles [\[edit\]](#)

A rotation in 3D space can be represented numerically with [matrices](#) in several ways. One of these representations is:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

An example worth examining happens when $\beta = \frac{\pi}{2}$. Knowing that $\cos \frac{\pi}{2} = 0$ and $\sin \frac{\pi}{2} = 1$, the above expression becomes equal to:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Carrying out [matrix multiplication](#):

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin \alpha & \cos \alpha & 0 \\ -\cos \alpha & \sin \alpha & 0 \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ \sin \alpha \cos \gamma + \cos \alpha \sin \gamma & -\sin \alpha \sin \gamma + \cos \alpha \cos \gamma & 0 \\ -\cos \alpha \cos \gamma + \sin \alpha \sin \gamma & \cos \alpha \sin \gamma + \sin \alpha \cos \gamma & 0 \end{bmatrix}$$

And finally using the [trigonometry formulas](#):

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\alpha + \gamma) & \cos(\alpha + \gamma) & 0 \\ -\cos(\alpha + \gamma) & \sin(\alpha + \gamma) & 0 \end{bmatrix}$$

Changing the values of α and γ in the above matrix has the same effects: the rotation angle $\alpha + \gamma$ changes, but the rotation axis remains in the Z direction: the last column and the first row in the matrix won't change. The only solution for α and γ to recover different roles is to change β .

Quaternion 四元數

A quaternion :

$$q = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$$

scalar
part

vector part

$$i^2 = j^2 = k^2 = ijk = -1$$

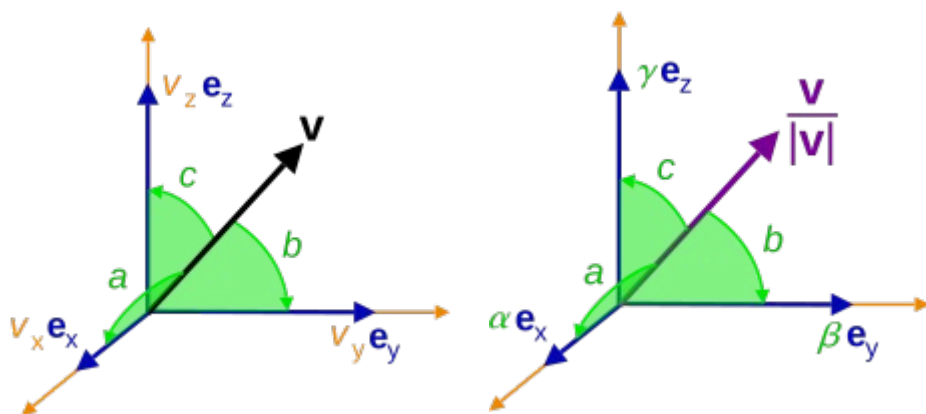
$$ij = k$$

$$ji = -k$$



Here as he walked by on the
16th of October 1843, Sir
William Rowan Hamilton in a
flash of genius discovered the
fundamental formula for
quaternion multiplication

Quaternion 四元數

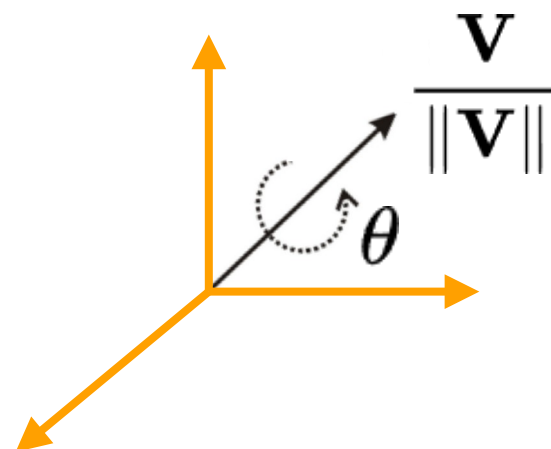


$$\mathbf{v} = v_x \mathbf{e}_x + v_y \mathbf{e}_y + v_z \mathbf{e}_z$$

$$\cos a = \frac{\mathbf{v} \cdot \mathbf{e}_x}{\|\mathbf{v}\|} = \frac{v_x}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

$$\cos b = \frac{\mathbf{v} \cdot \mathbf{e}_y}{\|\mathbf{v}\|} = \frac{v_y}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

$$\cos c = \frac{\mathbf{v} \cdot \mathbf{e}_z}{\|\mathbf{v}\|} = \frac{v_z}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$



$$q = [q_0, q_1, q_2, q_3]$$

$$q_0 = \cos(\theta/2)$$

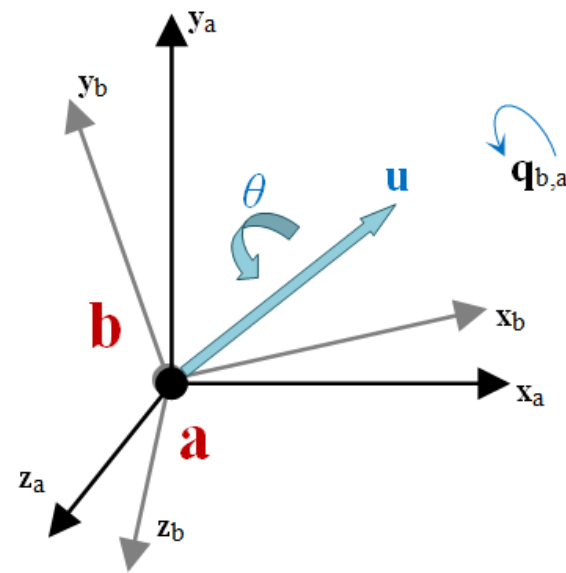
$$q_1 = \sin(\theta/2) \cos(a)$$

$$q_2 = \sin(\theta/2) \cos(b)$$

$$q_3 = \sin(\theta/2) \cos(c)$$

Quaternion 四元數

三度空間的一個向量，可以表示為一個純四元數，也就是沒有實數部份的四元數： $q_A = 0 + xi + yj + zk$ 一個旋轉可以用一個絕對值為 1 的四元數 q_R 表示， $|q_R| = 1$ 。一個向量 q_A ，從座標系 A 轉動到座標系 B, q_B ，可以用共軛運算來完成。



$$\begin{aligned} q_B &= q_R q_A q_R^* = (q_0 + q_1i + q_2j + q_3k)(xi + yj + zk)(q_0 - q_1i - q_2j - q_3k) \\ &= (x(q_0^2 + q_1^2 - q_2^2 - q_3^2) + 2y(q_1q_2 - q_0q_3) + 2z(q_0q_2 + q_1q_3))i + \\ &\quad (2x(q_0q_3 + q_1q_2) + y(q_0^2 - q_1^2 + q_2^2 - q_3^2) + 2z(q_2q_3 - q_0q_1))j + \\ &\quad (2x(q_1q_3 - q_0q_2) + 2y(q_0q_1 + q_2q_3) + z(q_0^2 - q_1^2 - q_2^2 + q_3^2))k. \end{aligned}$$

Quaternion 四元數

$$M = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_0q_2 + q_1q_3) \\ 2(q_0q_3 + q_1q_2) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_0q_1 + q_2q_3) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

$$|q| = q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1 \quad \Rightarrow \quad M = 2 \cdot \begin{bmatrix} q_0^2 + q_1^2 - 0.5 & q_1q_2 - q_0q_3 & q_0q_2 + q_1q_3 \\ q_0q_3 + q_1q_2 & q_0^2 + q_2^2 - 0.5 & q_2q_3 - q_0q_1 \\ q_1q_3 - q_0q_2 & q_0q_1 + q_2q_3 & q_0^2 + q_3^2 - 0.5 \end{bmatrix}$$

$$\begin{aligned} \text{Trace}(M) &= M_{11} + M_{22} + M_{33} \\ &= 2(3q_0^2 + q_1^2 + q_2^2 + q_3^2 - 1.5) \\ &= 2(3q_0^2 + (1 - q_0^2) - 1.5) \\ &= 2(2q_0^2 - 0.5) \\ &= 4q_0^2 - 1. \end{aligned}$$

$$M_{11} = 2(q_0^2 + q_1^2 - 0.5) = 2 \left(\frac{\text{Trace}(M) + 1}{4} + q_1^2 - 0.5 \right)$$

.....

$$|q_0| = \sqrt{\frac{\text{Trace}(M) + 1}{4}}$$

$$|q_1| = \sqrt{\frac{M_{11}}{2} + \frac{1 - \text{Trace}(M)}{4}}$$

$$|q_2| = \sqrt{\frac{M_{22}}{2} + \frac{1 - \text{Trace}(M)}{4}}$$

$$|q_3| = \sqrt{\frac{M_{33}}{2} + \frac{1 - \text{Trace}(M)}{4}}$$

四元數導算範例，Z 旋轉軸

Given a rotation by an angle, the corresponding quaternion can be computed. Consider the rotation ψ around the z-axis:

繞 Z 軸旋轉矩陣 :
$$M_\psi = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

From the formulas in the previous section and using that $\text{Trace}(M) = 2 \cos \psi + 1$:

$$|q_0| = \sqrt{\frac{2 \cos \psi + 1 + 1}{4}} = \sqrt{\frac{1 + \cos \psi}{2}} = \cos \frac{\psi}{2},$$

求解對應
四元數 :

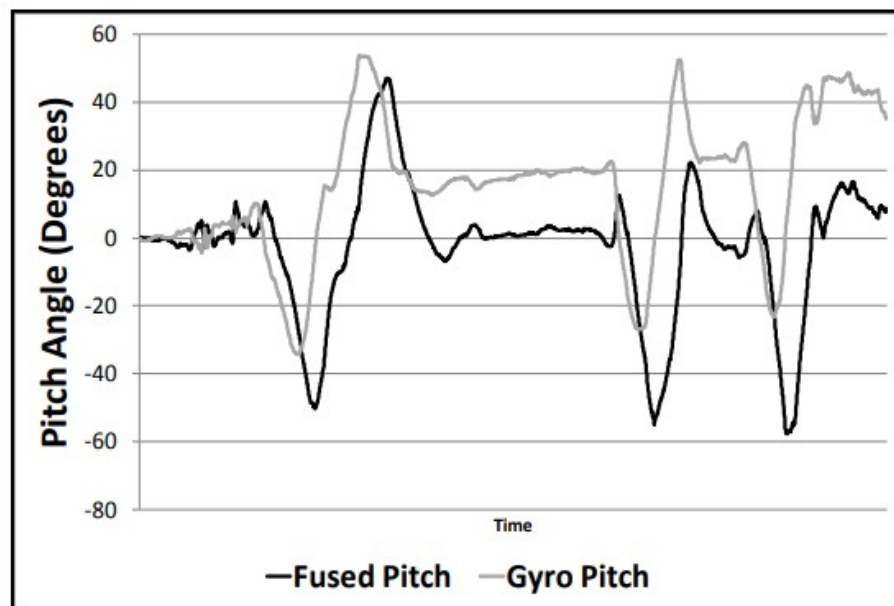
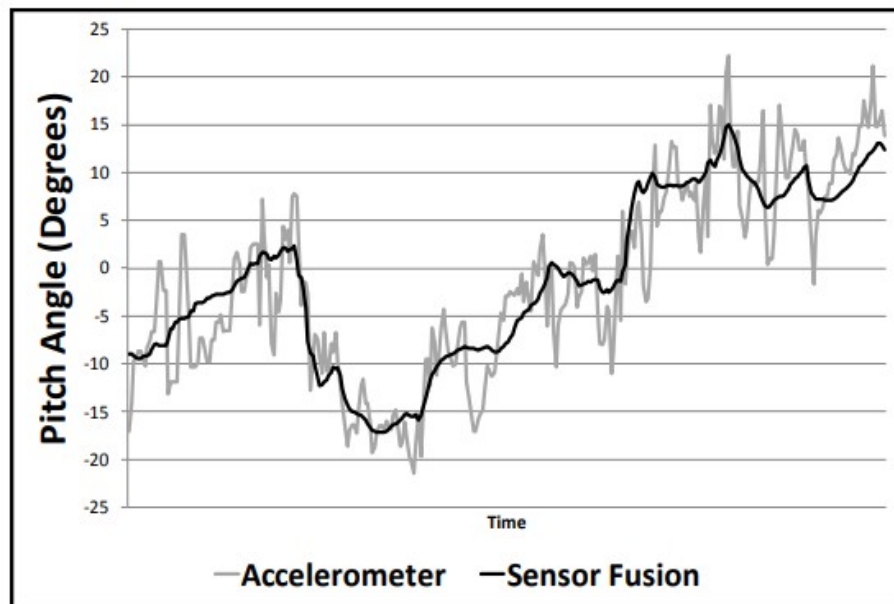
$$|q_1| = |q_2| = \sqrt{\frac{\cos \psi}{2} + \frac{1 - (2 \cos \psi + 1)}{4}} = 0,$$

$$|q_3| = \sqrt{\frac{1}{2} + \frac{1 - (2 \cos \psi + 1)}{4}} = \sqrt{\frac{1 - \cos \psi}{2}} = \sin \frac{\psi}{2}.$$

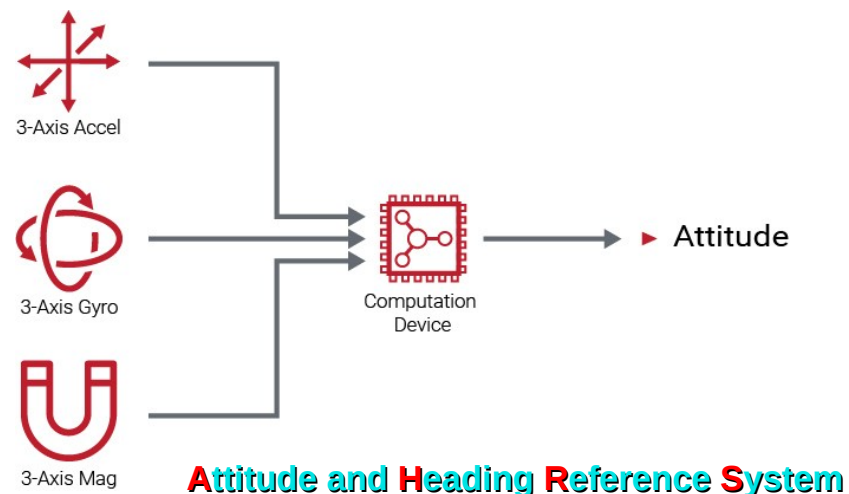
Therefore:

$$q_\psi = \cos \frac{\psi}{2} + k \sin \frac{\psi}{2}.$$

處理感測器誤差 - Sensor Fusion



加速計易受雜訊干擾，而陀螺儀隨著量測時間數值有漂移現象。加速計搭配陀螺儀，輔以讀值融合演算法，是常用的處理方法。



飛行姿態控制 - 姿態更新

理想姿態更新可僅由陀螺儀計算（角速度積分）

$$\theta_{i,t} = \theta_{i,t_0} + \int_{t_0}^t \omega_i dt \quad i = x, y, z$$

實際姿態更新由於的困擾使用四元數更新

$$q_t = q_0 + \int_{t_0}^t \omega dt \quad \longrightarrow \quad \frac{dq}{dt} = \frac{1}{2}(\omega + \omega_{adjust})q$$

Quaternion : 姿態 ↔ 座標轉換

四元數導數

方向（姿態）可以用四元數 $q(t)$ 在時間 t 來描述，經過時間 Δt 的姿態更新為 $q(t + \Delta t)$ ，這是在 Δt 時間執行的一個旋轉變化 Δq 。

$$\hat{u} = \frac{\hat{\omega}}{\|\omega\|}$$

$$\Delta \hat{q} = \cos \frac{\theta}{2} + \hat{u} \sin \frac{\theta}{2} = \cos \frac{\|\omega\| \Delta t}{2} + \frac{\hat{\omega}}{\|\omega\|} \sin \frac{\|\omega\| \Delta t}{2}$$

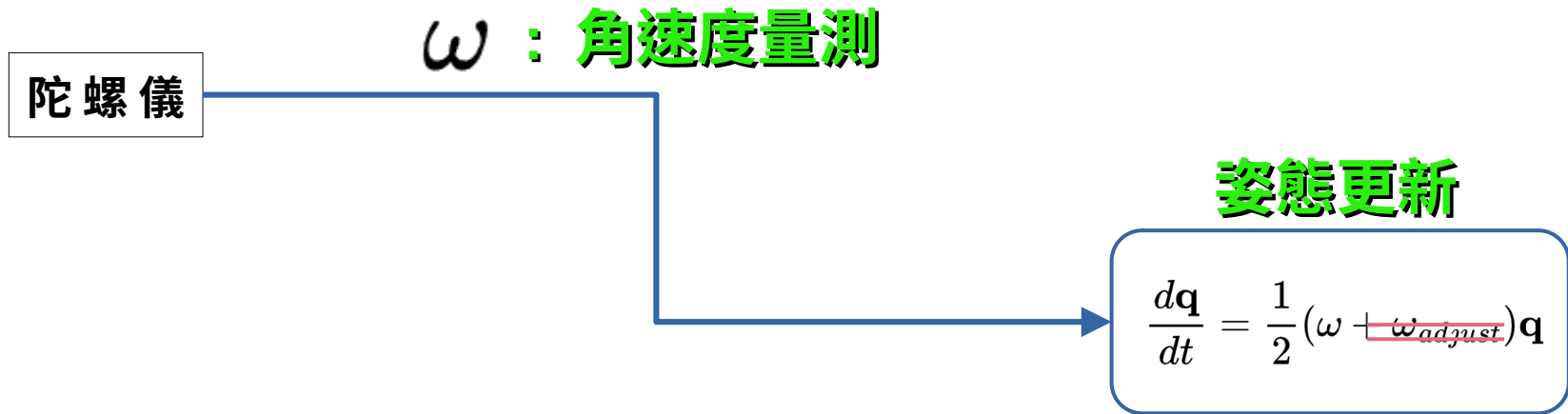
$$\hat{q}(t + \Delta t) = \Delta \hat{q} \bullet \hat{q}(t)$$

$$\begin{aligned} \hat{q}(t + \Delta t) - \hat{q}(t) &= \left(\cos \frac{\|\omega\| \Delta t}{2} + \frac{\hat{\omega}}{\|\omega\|} \sin \frac{\|\omega\| \Delta t}{2} \right) \hat{q} - \hat{q} \\ &= \left(-2 \sin^2 \frac{\|\omega\| \Delta t}{4} + \frac{\hat{\omega}}{\|\omega\|} \sin \frac{\|\omega\| \Delta t}{2} \right) \hat{q} \end{aligned}$$

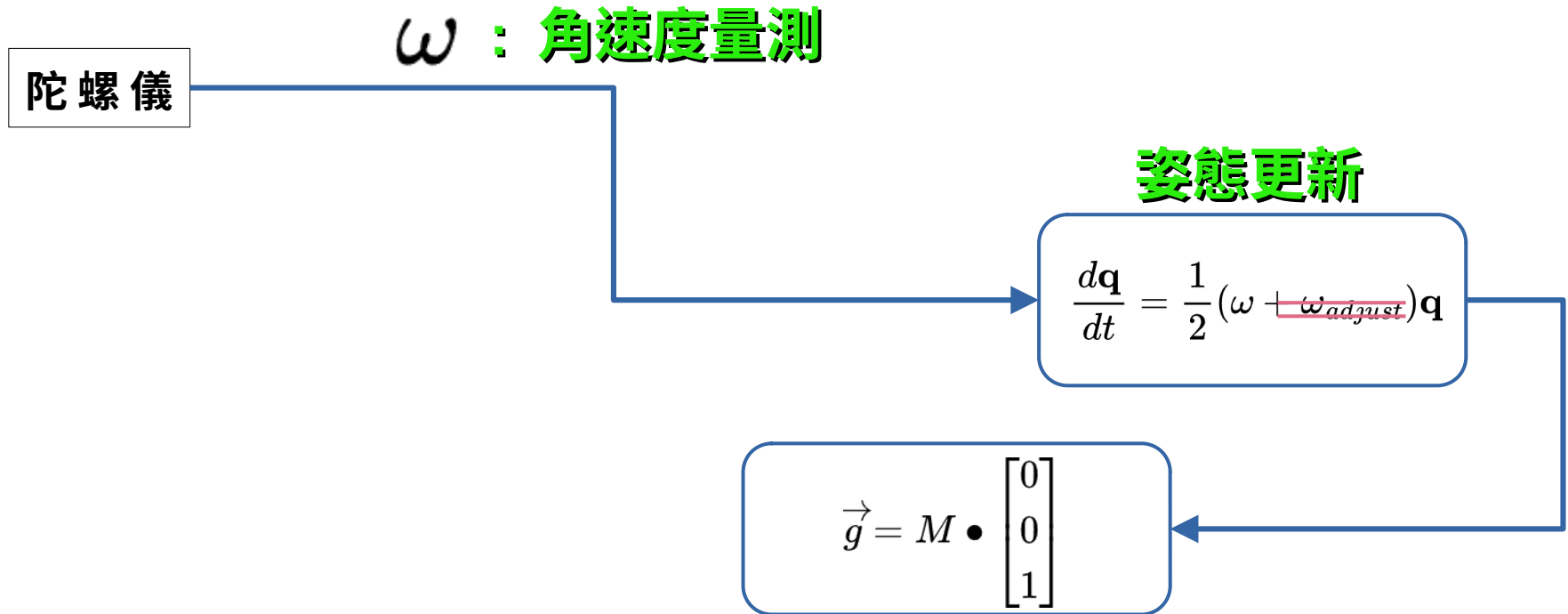
四元數導數

$$\begin{aligned}\frac{d\hat{q}}{dt} &= \lim_{\Delta t \rightarrow 0} \frac{\hat{q}(t + \Delta t) - \hat{q}(t)}{\Delta t} \\&= \lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \left(-2 \sin^2 \frac{\|\omega\| \Delta t}{4} + \frac{\hat{\omega}}{\|\omega\|} \sin \frac{\|\omega\| \Delta t}{2} \right) \hat{q} \\&= \frac{\hat{\omega}}{\|\omega\|} \lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \sin \left(\frac{\|\omega\| \Delta t}{2} \right) \hat{q} \\&= \frac{\hat{\omega}}{\|\omega\|} \frac{d}{dt} \sin \left(\frac{\|\omega\|}{2} t \right) \Big|_{t=0} \hat{q} \\&= \frac{1}{2} \hat{\omega} \hat{q}\end{aligned}$$

Sensor Fusion – Mahony 演算法 ①



Sensor Fusion – Mahony 演算法 ②

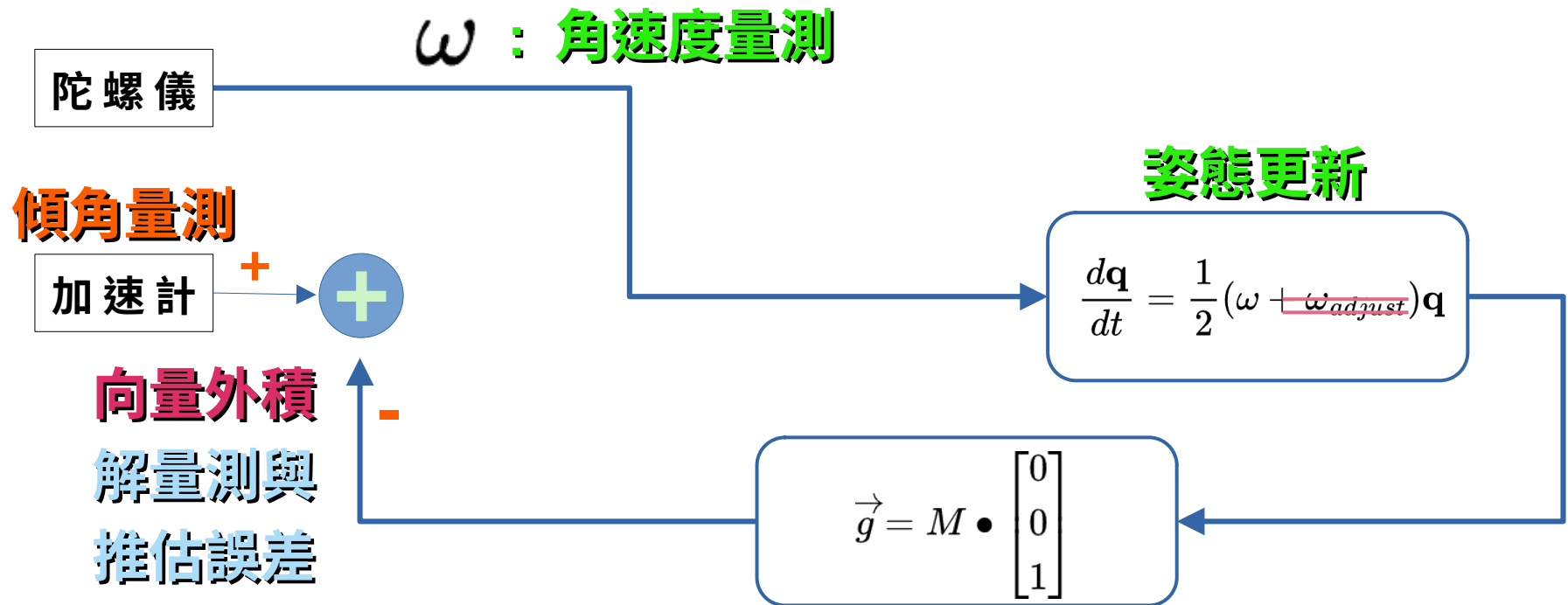


將以地面為參考座標的重力向量 (0,0,1)
轉換到以無人機為參考座標的向量

傾角推估

$$M = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_0q_2 + q_1q_3) \\ 2(q_0q_3 + q_1q_2) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_0q_1 + q_2q_3) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

Sensor Fusion – Mahony 演算法 ③

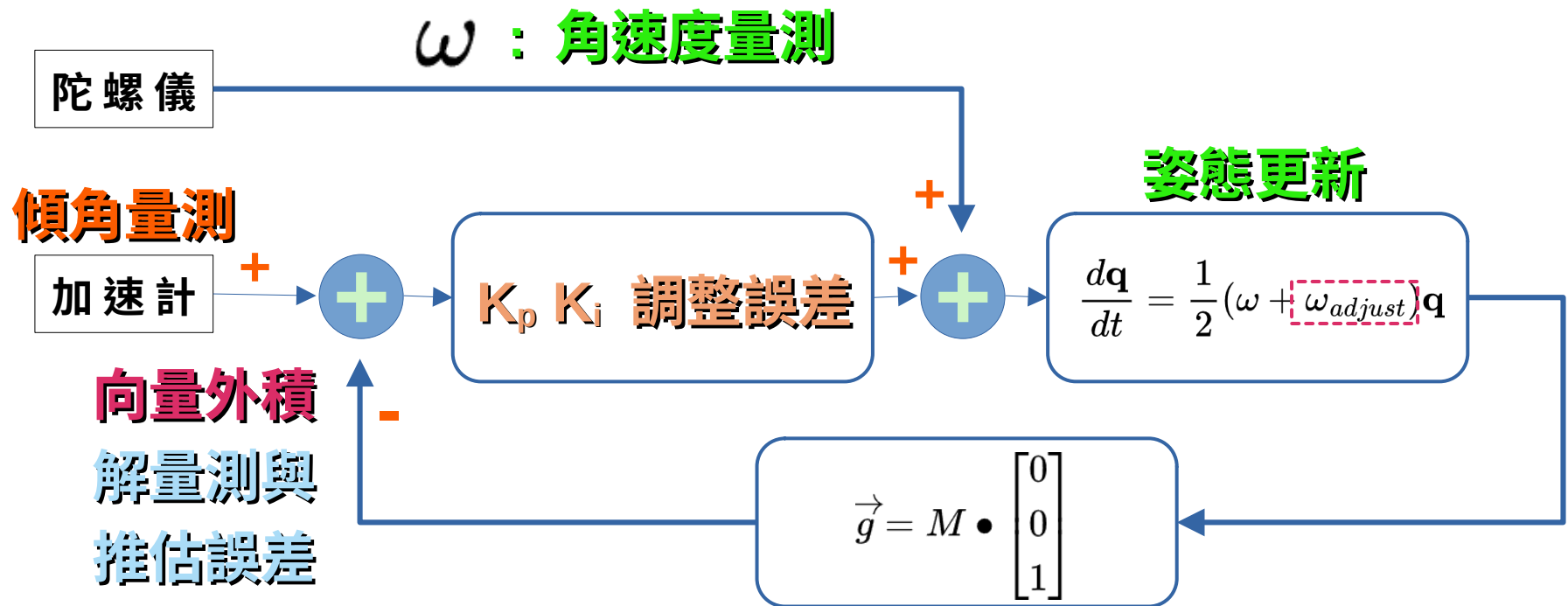


將以地面為參考座標的重力向量 (0,0,1)
轉換到以無人機為參考座標的向量

傾角推估

$$M = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_0q_2 + q_1q_3) \\ 2(q_0q_3 + q_1q_2) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_0q_1 + q_2q_3) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

Sensor Fusion – Mahony 演算法 ④



將以地面為參考座標的重力向量 (0,0,1)
轉換到以無人機為參考座標的向量

傾角推估

$$M = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_0q_2 + q_1q_3) \\ 2(q_0q_3 + q_1q_2) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_0q_1 + q_2q_3) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

Sensor Fusion – Mahony 演算法

1. 初始化 Quaternion, $q = (1, 0, 0, 0)$
2. 將單位重力向量 $g_{\text{Earth}} = [0, 0, 1]_{\text{Earth}}$, 經由 q 轉換為無人機的參考座標 , $g_{\text{body}} = M_q \cdot [0, 0, 1]_{\text{Earth}}^T$
3. 利用向量外積解出 , 無人機加速計讀值 a_{body} 和 g_{body} 的誤差 , $\text{Error} = a_{\text{body}} \otimes g_{\text{body}}$
4. 用 PI 調整角速度 $\rightarrow w_{\text{adjust}} = K_p \cdot \text{Error} + K_i \cdot \int \text{Error}$
5. 更新 Quaternion, $q_{t+1} = q_t + (\frac{1}{2})q_t(w + w_{\text{adjust}})dt$
6. 繼續重複步驟 2. ~ 5.

Mahony 演算法 - C 程式範例

Mahony Filter

```
void ahrs_fusion_ag(AxesRaw_TypeDef_Float *acc,
                  AxesRaw_TypeDef_Float *gyro,
                  AHRS_State_TypeDef *ahrs)
{
    float axf, ayf, azf, gxf, gyf, gzf;
    float norm;
    float vx, vy, vz;
    float ex, ey, ez;
    float q0q0, q0q1, q0q2, q0q3, q1q1, q1q2, q1q3, q2q2, q2q3, q3q3;
    float halfT;

    if(gTHR<MIN_THR)
    {
        ahrs_kp = AHRS_KP_BIG;
    }
    else
    {
        ahrs_kp = AHRS_KP_NORM;
    }

    axf = acc->AXIS_X;
    ayf = acc->AXIS_Y;
    azf = acc->AXIS_Z;

    // mdps convert to rad/s
    gxf = gyro->AXIS_X * COE_MDPS_TO_RADPS;
    gyf = gyro->AXIS_Y * COE_MDPS_TO_RADPS;
    gzf = gyro->AXIS_Z * COE_MDPS_TO_RADPS;

    // auxiliary variables to reduce number of repeated operations
    q0q0 = q0*q0;
    q0q1 = q0*q1;
    q0q2 = q0*q2;
    q0q3 = q0*q3;
    q1q1 = q1*q1;
    q1q2 = q1*q2;
    q1q3 = q1*q3;
    q2q2 = q2*q2;
    q2q3 = q2*q3;
    q3q3 = q3*q3;
```

```
// normalise the accelerometer measurement
norm = invSqrt(axf*axf+ayf*ayf+azf*azf);
```

```
axf = axf * norm;
ayf = ayf * norm;
azf = azf * norm;
```

← 傾角量測

```
// estimated direction of gravity and flux (v and w)
```

```
vx = 2*(q1q3 - q0q2);
vy = 2*(q0q1 + q2q3);
vz = q0q0 - q1q1 - q2q2 + q3q3;
```

← 傾角推估

```
ex = (ayf*vz - azf*vy);
ey = (azf*vx - axf*vz);
ez = (axf*vy - ayf*vx);
```

← 向量外積

```
// integral error scaled integral gain
exInt = exInt + ex*AHRS_KI*SENSOR_SAMPLING_TIME;
eyInt = eyInt + ey*AHRS_KI*SENSOR_SAMPLING_TIME;
ezInt = ezInt + ez*AHRS_KI*SENSOR_SAMPLING_TIME;
```

```
// adjusted gyroscope measurements
```

```
gxf = gxf + ahrs_kp*ex + exInt;
gyf = gyf + ahrs_kp*ey + eyInt;
gzf = gzf + ahrs_kp*ez + ezInt;
```

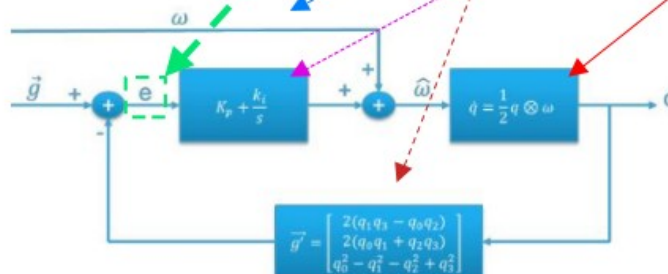
```
// integrate quaternion rate and normalise
```

```
halfT = 0.5*SENSOR_SAMPLING_TIME;
q0 = q0 + (-q1*gxf - q2*gyf - q3*gzf)*halfT;
q1 = q1 + (q0*gxf + q2*gzf - q3*gyf)*halfT;
q2 = q2 + (q0*gyf - q1*gzf + q3*gxf)*halfT;
q3 = q3 + (q0*gzf + q1*gyf - q2*gxf)*halfT;
```

```
// normalise quaternion
```

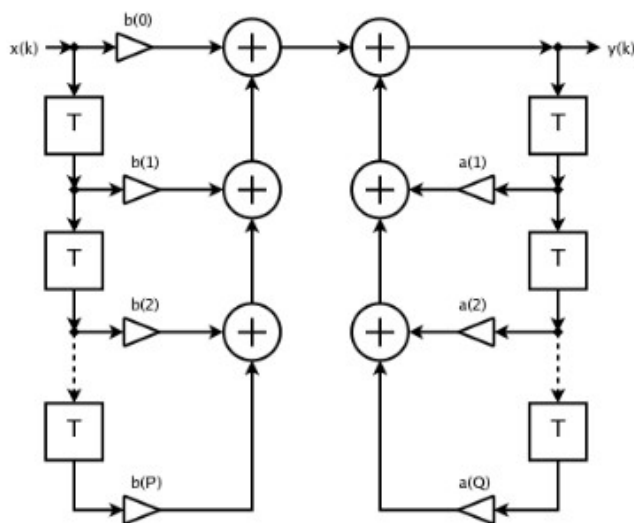
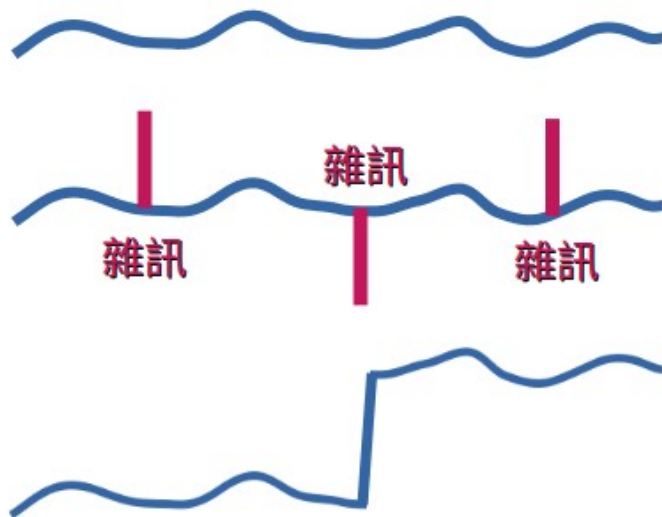
```
norm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
q0 *= norm;
q1 *= norm;
q2 *= norm;
q3 *= norm;
```

```
ahrs->q.q0 = q0;
ahrs->q.q1 = q1;
ahrs->q.q2 = q2;
ahrs->q.q3 = q3;
```



處理雜訊，IIR 是什麼？

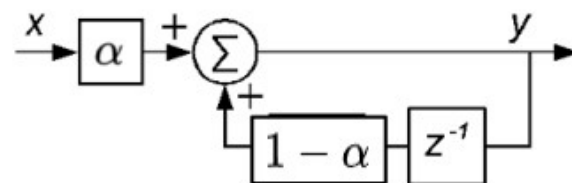
原始感測器的讀值，在進入感測器融合演算法前，通常會處理雜訊



IIR : Infinite impulse response

$$y_n = (1 - \alpha)y_{n-1} + \alpha x_n$$

$$0 \leq \alpha \leq 1$$



當 α 較小的時候，表示輸出的訊號 y_n 較依賴之前的訊號 y_{n-1} ，而不太採信剛收到的訊號 x_n ，因此突發短暫的雜訊就會被抑制。四軸無人機通常會用到三級 IIR。